Day 7 : State management

1. Introduction to State Management

Flutter applications are built using widgets. Some widgets display static information, while others need to update their content when data changes. The data that can change during the lifetime of an application is called **State**.

State Management is the technique used to manage and update this changing data so that the user interface (UI) reflects the latest information.

Example:

Consider a counter application.

- Initial value = 0
- User clicks the "+" button
- Value changes to 1
- UI updates automatically

The counter value is the **state** of the application.

Without state management, the UI would not know when to update and users would continue seeing outdated information.

2. What is State?

State is any information that can change while the application is running.

Examples of State

- Counter value
- Username entered in a form
- Selected checkbox
- Current theme (Light/Dark)
- Todo list items
- Quiz score
- Shopping cart items

Non-State Data

Data that never changes during runtime is not considered state.

3. Why Do We Need State Management?

1. Update the UI Automatically

Whenever data changes, Flutter rebuilds the required widgets.

2. Improve User Experience

Users immediately see updated information.

3. Handle Dynamic Data

Apps become interactive and responsive.

4. Organize Application Data

Makes code cleaner and easier to maintain.

5. Build Real-World Applications

Almost every modern app uses state management.

Examples:

- Instagram likes
- WhatsApp messages
- Shopping carts
- Banking apps

4. Types of State in Flutter

Flutter mainly has two types of state.

1. Ephemeral (Local) State

Ephemeral State is temporary state used by a single widget.

Characteristics

- Exists only inside one widget.
- Short-lived.
- Managed using setState().

Examples

- Counter value
- Checkbox selection
- Password visibility toggle

Advantages

- Easy to implement
- Suitable for small applications

```
int count = 0;
```

2. App-Wide State

App-Wide State is shared across multiple widgets and screens.

Characteristics

- Used throughout the application.
- Multiple widgets can access it.
- Managed using Provider, Riverpod, Bloc, etc.

Examples

- User login information
- Shopping cart
- Theme settings
- Language preferences

Advantages

- Better for large applications.
- Easy data sharing between screens

5. StatefulWidget

Flutter provides two major widget types:

StatelessWidget

A widget whose data never changes.

6. Stateful WIdget

A widget whose data can change during runtime.

Structure of StatefulWidget

A StatefulWidget consists of two classes:

Widget Class

```
class CounterPage extends StatefulWidget {
  @override
  _CounterPageState createState() => _CounterPageState();
}
```

State Class

```
class _CounterPageState extends State<CounterPage> {
  int count = 0;

  @override
  Widget build(BuildContext context) {
    return Text("$count");
  }
}
```

7. createState() Method

The createState() method creates the State object for a StatefulWidget.

Purpose

- Connects widget and state.
- Allows Flutter to track changes.

Example:

```
@override
_CounterPageState createState() => _CounterPageState();
```

8. State Lifecycle

1. createState()

Called when widget is created.

Purpose

Creates state object.

2. initState()

Called once when widget is inserted into widget tree.

Purpose

Initialization tasks.

Examples

- API calls
- Variable initialization
- Controller setup

```
@override
void initState() {
  super.initState();
}
```

3. build()

Builds the user interface.

Called whenever state changes.

Purpose

Displays widgets on screen.

```
Widget build(BuildContext context)
```

4. didUpdateWidget()

Called when parent widget updates properties.

Purpose

Respond to configuration changes.

```
@override
void didUpdateWidget(oldWidget) {
  super.didUpdateWidget(oldWidget);
}
```

5. dispose()

Called when widget is removed.

Purpose

Free resources.

Examples:

- Close streams
- Dispose controllers

```
@override
void dispose() {
  controller.dispose();
  super.dispose();
}
```

9 . setState()

The most important method in basic Flutter state management.

Definition

setState() tells Flutter that state has changed and UI needs rebuilding.

```
setState(() {
  count++;
});
```

```
ElevatedButton(
  onPressed: () {
    setState(() {
      count++;
    });
  },
  child: Text("Increment"),
)
```

b

10 . Form State Management

Forms are common in applications.
Examples:

- Login Form
- Registration Form
- Contact Form

TextEditingController

- Used to control and retrieve text field data.
- **Example**

```
TextEditingController nameController =
    TextEditingController();
```

11. Managing Lists

Many applications use lists.
Examples:

- Contacts
- Todo items
- Products

1. Add Items

```
setState(() {
  items.add("New Item");
});
```

2. Remove items

```
setState(() {
  items.removeAt(index);
});
```

3. Update Items

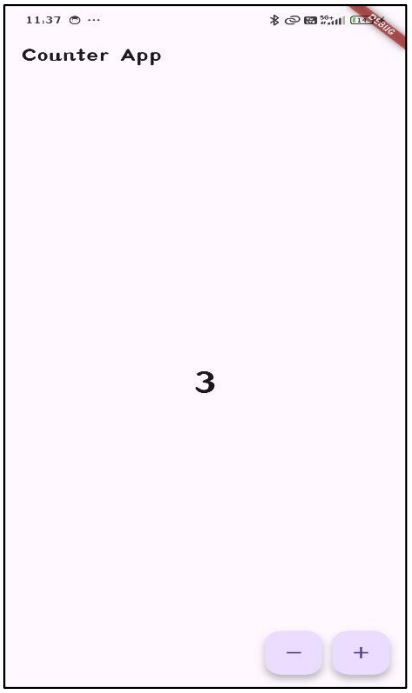
```
setState(() {
  items[index] = "Updated Item";
});
```

4. Search and Filter

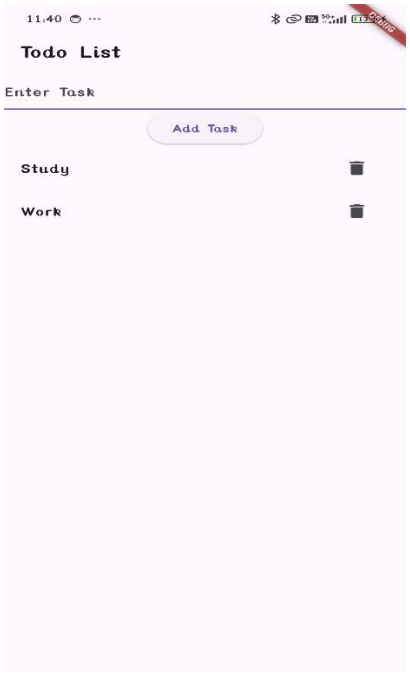
```
items.where(
  (item) => item.contains(searchText)
);
```

Code :

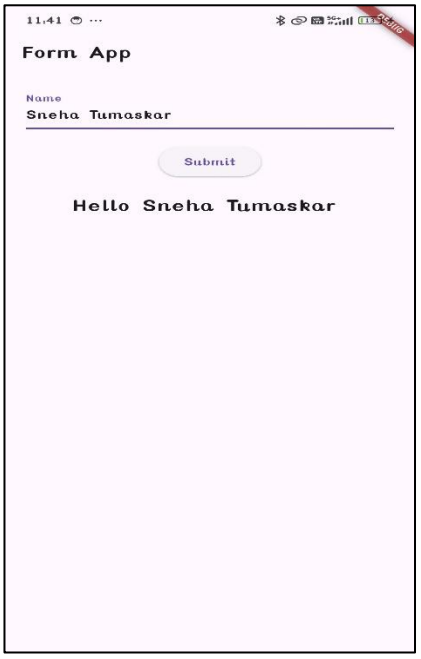
Counter :



To Do list :



Form :


11:41
Form App
Name
Sneha Tumaskar
Submit
Hello Sneha Tumaskar